# Software Requirements Specification

**for**

# CodeVenture

**Version 1.0 approved**

**Prepared by Sahab, Nicolas, Taraaf, Yew Yi**

**14/08/2023**

*Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.*

# Table of Contents

**1. Introduction....1**
1.1 Purpose....1
1.2 Problem Statement....1
1.3 References....2
**2. Stakeholders and Actors of the System....3**
2.1 Stakeholders....3
2.2 Stakeholder Onion Model....5
2.3 Influence-Importance Matrix....6
2.4 Goals and Roles of Each Stakeholder....7
2.4 Persona Representation....9
2.5 Stakeholder Interview Outcomes....13
**3. System Features....14**
3.1 User Authentication and Profile Management....14
3.2 Learning Modules....15
3.3 Challenges and Quizzes....16
3.4 Progress Tracking....17
3.5 User Interface....17
3.6 Stakeholder Views....18
3.7 Ethics and Accessibility....18
3.8 Feedback and Support....19
3.9 Performance and Reliability....19
3.10 Content Management....20
**4. Non-Functional Requirements....22**
4.1 Performance....22
4.2 Usability....22
4.3 Reliability....22
4.4 Security....23
4.5 Availability....23
4.6 Maintainability....23
4.7 Scalability....23
4.8 Ethics and Inclusivity....23
4.9 Compatibility....25
**5. Acceptance Criteria....26**
5.1 User Registration....26
5.2 Password Recovery....26

5.3 Profile Management....26
5.4 Learning Modules....26
5.5 Progress Tracking....26
5.6 User Interface....27
5.8 Ethics and Accessibility....27
5.9 Performance and Reliability....27
5.10 Feedback and Support....27
5.11 Content Management....28
5.12 Community and Collaboration....28
5.13 Security....28
5.14 Compatibility....28
5.15 Challenge and Quiz....28
**6. Requirements Quality Analysis....29**
6.1 Key Quality Attributes to Assess....29
6.2 Common Requirements Defects....29
6.3 Analysis and Improvements....29
**7. Document Quality....31**
7.1 Document Structure....31
7.2 Visual Aids....31
7.3 Requirement Categorisation....31
7.4 Examples and Scenarios....31
7.5 Versioning....32
7.6 Feedback Mechanism....32
7.7 Stakeholder Profiles....32
7.8 Document Formatting....32
7.9 Document Summary....32

# 1. Introduction

## 1.1 Purpose

In the digital age, programming has grown beyond a niche skill to a foundational literacy. Recognising the significance of this trend, our team embarked on the ambitious project of creating CodeVenture, an interactive game platform aimed at inspiring young learners to embrace the world of Python programming. This requirements specification document outlines the key functionalities and attributes that CodeVenture will offer, from user account creation to learning modules and progress tracking. Driven by the feedback of educators, parents, and learners, we have encapsulated a vision that prioritises engagement, inclusivity, and ethical considerations. The subsequent sections will delve deep into functional, non-functional requirements and the acceptance criteria set for the system. Each section is meticulously designed to ensure that the end product not only resonates with its target audience but also stands out in the vast ed-tech marketplace.

## 1.2 Problem Statement

Coding has become an essential skill with the emergence of digital technology. Young students are frequently exposed to a variety of programming languages, with Python being one of the most user-friendly. Many learning platforms, however, fail to capture the attention and enthusiasm of these young learners, which is essential for long-term and sustained engagement.

Many existing platforms provide a one-size-fits-all approach to teaching Python, which may not address the special requirements and interests of young learners as it is too generic of a learning platform. Furthermore, Traditional coding tutorials and platforms frequently lack dynamic and interesting content, making learning feel more like a chore for young students, thus creating a lack of engagement.

The idea is to create CodeVenture, a game that takes young learners on an interactive and enjoyable trip into the world of programming in Python. This game seeks to not only teach but also inspire and challenge learners, keeping them interested in their coding experiences.

## 1.3 References

Stakeholder Matrix Template: https://online.visual-paradigm.com

Stakeholder Onion Model: https://www.lucidchart.com

Persona Templates: https://www.canva.com/

# 2. Stakeholders and Actors of the System

## 2.1 Stakeholders

### Young Learners

The primary users of CodeVenture. Young individuals, typically aged between 8 to 16 who are interested in learning programming in Python. Their main goal is to learn and understand the fundamentals of programming and Python and apply the knowledge gained to real-world applications, all whilst enjoying an interactive and fun experience. The platform should therefore be age-appropriate, engaging, and not too challenging that it overwhelms them.

### Parents

Guardians who are either funding the platform for their kids or monitoring their progress. Their goal is generally to ensure that the platform provides value for the money and to track the progress and safety of their child. The platform should be safe, devoid of inappropriate content, and avoid mechanisms such as unchecked in-app purchases that lead to undue expenses, for example.

### Educators

Teachers, tutors, or coaches who might integrate CodeVenture into their curriculum or recommend it as a supplementary tool. The primary goal is to provide an engaging teaching method and track student progress, as well as easily assign tasks. The content should align with educational standards and be adjustable to different learning paces.

### Managers/Project Owners

Individuals or a group responsible for the strategic direction, funding, and overall success of the CodeVenture project. They might represent the company, organisation, or investor group that initiated the project.

### Developers

The dedicated technical team entrusted with the development, testing, and deployment of the CodeVenture platform. Their objective is to Create a stable, scalable, and efficient platform and regularly update with new content and features. They require precise requirements and feedback. The system's architecture should allow for future scalability and maintenance.

### Admins

The administrative personnel oversees user management, system health, and handles issues like user reports. Their main role is to ensure a smooth, hassle-free user experience, and swiftly address any system or user issues. They require tools and interfaces to monitor platform health, user behaviour, and manage content.

### School Administrators

Management of educational institutions that might want to integrate the platform on a larger scale. Their main goal is to evaluate the platform's effectiveness for institution-wide adoption and obtain bulk or institutional discounts. They need reports on the platform's effectiveness, ease of integration into existing systems, and financial feasibility.

### Advertisers

As potential sponsors, advertisers are interested in leveraging the CodeVenture platform to showcase their ads or promotions to the platform's user base. Their primary objectives revolve around achieving successful advertising campaigns that precisely target a youthful demographic, which aligns with the platform's user profile. They require data on user engagement, click-through rates, and return on investment.

### Educational Policy Makers

Individuals or groups that set educational standards the platform might need to adhere to.
Their primary objective includes ensuring that tools like CodeVenture meet educational quality and safety standards. They would want transparency in the platform's content creation process, effectiveness metrics, and safety measures.

## 2.2 Stakeholder Onion Model

- Core: Managers/Project Owners, Developers, Admins.
- Primary: Young Learners, Parents, Educators.
- Secondary: School Administrators, Advertisers, Educational Policy Makers.
- Periphery: Potential future users, the general public, competitors.

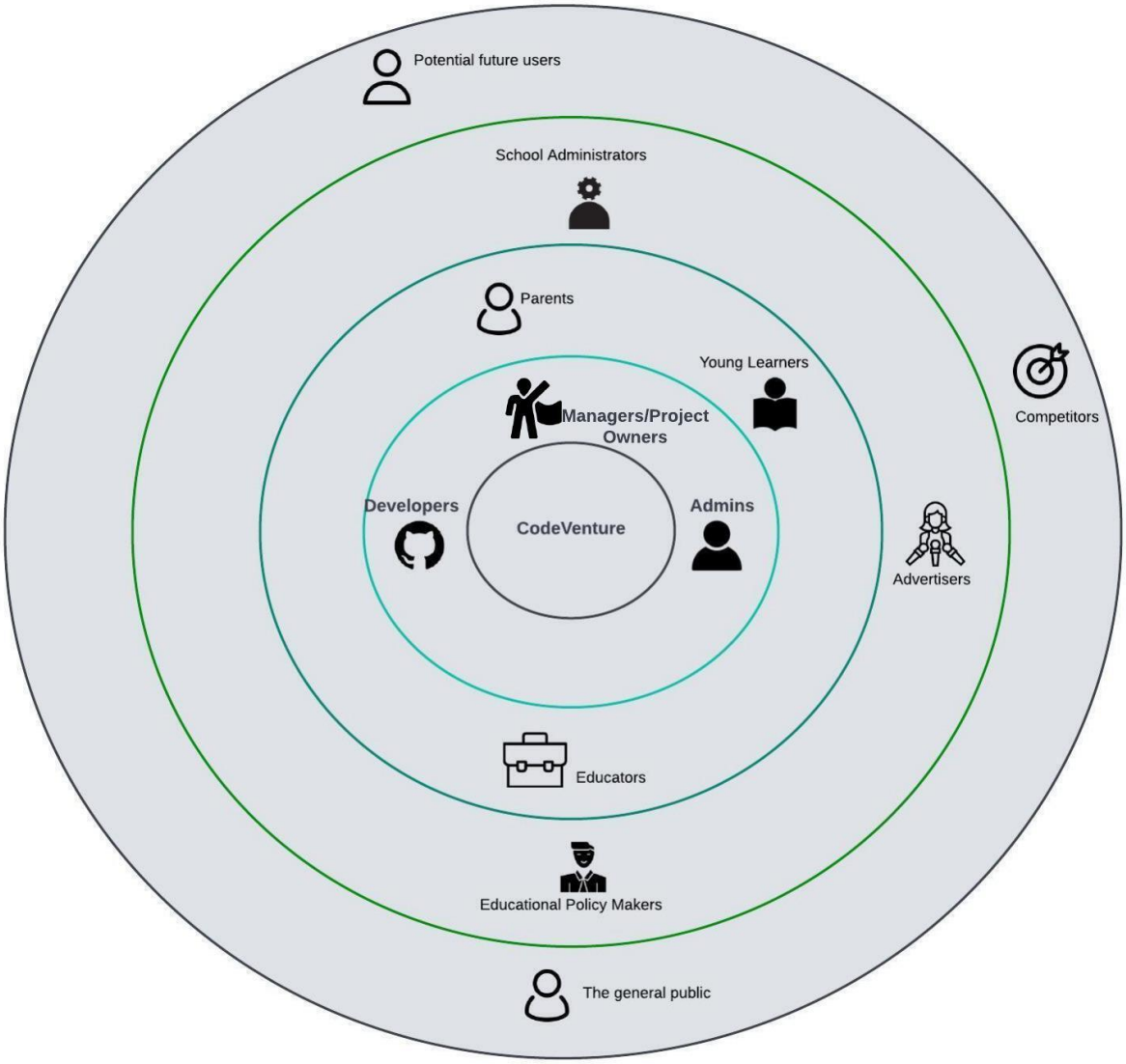



*Figure 2.2.1 - Showing our stakeholder onion model*

## 2.3 Influence-Importance Matrix

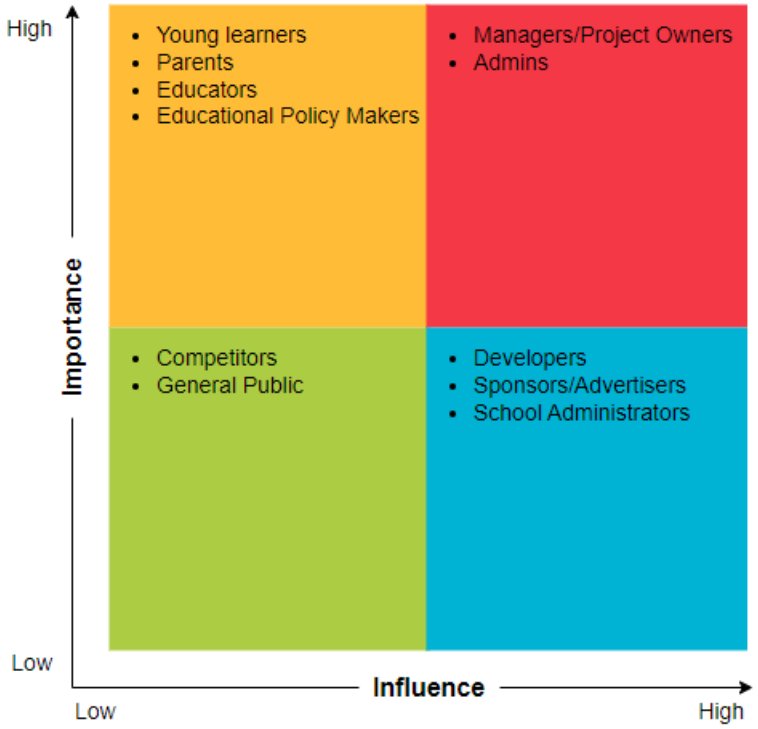



*Figure 2.3.1 - Showing our Influence-Importance Matrix chart*

**High Importance, High Influence**

- Managers/Project Owners: overarching decision-making responsibilities.
- Admins: responsible for the system's smooth operation, user management, and potential content curation. Their decisions can influence user experience and overall system stability.

**High Importance, Low Influence**

- Young Learners (Users/Customers): the main user group whose experience and feedback are crucial.
- Parents: their approval and feedback can significantly influence younger users' engagement.
- Educators (Teachers/Tutors): they integrate CodeVenture into their curriculum, making them essential users. Their feedback can significantly shape the platform's utility in real-world educational settings.
- Educational Policy Makers: they have a stake in the content and approach but might not have direct system influence.

**Low Importance, High Influence**

- Developers: play a pivotal role in realising the system's vision.
- Advertisers: their interests can influence monetisation strategies and platform features.
- School Administrators: can influence platform adoption at the institutional level.

**Low Importance, Low Influence**

- Competitors: they monitor the platform's progression but don't influence its direction.
- General Public: hold a generalised interest without a direct stake in the platform's specifics.

## 2.4 Goals and Roles of Each Stakeholder

### Young Learners

Goal: Learn Python in an engaging manner.

Role: Actively engage with the platform's learning modules, challenges, and quizzes.

### Parents

Goal: Ensure their child's learning progress and safety.

Role: Monitor the progress of the young learner and possibly make purchasing decisions.

### Educators

Goal: Enhance their teaching methods and track student progress.

Role: Supplement their teaching with the platform and possibly set assignments or challenges.

### Managers/Project Owners

Goal: Ensure the project's success, both in terms of user adoption and financial returns.

Role: Set the strategic direction, allocate resources, make major decisions, and ensure alignment among all stakeholders.

### Admins

Goal: Ensure the smooth running of the platform.

Role: Handle user complaints, manage accounts, and oversee general platform health.

### Developers

Goal: Build and maintain a functional, user-friendly platform.
Role: Code, test, and deploy the system components.

### School Administrators

Goal: Integrate a valuable coding tool into the school curriculum and ensure it provides educational value.
Role: Decide on platform adoption within the educational institution, manage institutional accounts, and coordinate with educators to implement the system in the curriculum.

### Educational Policy Makers

Goal: Set guidelines and standards that educational tools, like CodeVenture, should adhere to.
Role: Evaluate the educational value of the platform, ensuring it aligns with national or regional educational standards and policies.

### Advertisers

Goal: Reach a targeted audience through the platform for marketing purposes.
Role: Collaborate with platform admins for advertisement placements, track the performance of their ads, and adjust marketing strategies based on user engagement.

### General Public

Goal: Gain awareness and possibly use the platform for individual learning or recommend it to others.
Role: They may provide feedback, testimonials, or reviews which can affect public perception of the platform.

## 2.4 Persona Representation



# MAYA SMITH

Young Learner

- Age: 11
- Student, 6th Grade

### CORE GOALS:

- To learn Python coding in an engaging manner.
- To create a simple game for a school project.
- To impress and engage her elder brother in coding.
- To integrate coding projects for her school's robotics club.

### WORKING SITUATION:

- Full-time student

### PLACE OF WORK:

- Primarily at school
- Secondarily at home

### SOFTWARE EXPERTISE:

- Basic knowledge of Scratch.
- Attempted code blocks

### CORE CHARACTERISTICS:

- Curious and eager to learn new things.
- Enthusiastic about creative activities, especially coding and robotics

### SINGULARITIES

- Combining interests in graphic novels, coding, and robotics.
- Ability to apply coding skills to creative projects

### MISCELLANEOUS:

- Enjoys hands-on activities and interactive learning.
- Comfortable with digital devices and online platforms.
- Values recognition from peers and family.

### TYPICAL CHALLENGES:

- Finding engaging and interactive learning resources for coding.
- Overcoming the fear of advanced content and being overwhelmed.
- Needing timely feedback and assistance as a beginner in coding.

### MAIN TASKS WITH SYSTEM SUPPORT

- Ensuring her children access enriching online platforms.
- Implementing parental controls for screen time and communication.
- Communicating directly with platform administrators and educators.

### MOST IMPORTANT TASKS:

- Learning Python concepts through interactive and engaging lessons
- Creating a simple game using Python coding for a school project
- Integrating coding skills into the school's robotics club projects.

### MOST IMPORTANT TASKS:

- Exploring unrelated topics on the CodeVenture platform
- Participating in activities that don't align with her coding and robotics interests.



# SARAH MITCHELL

Educator

## ABOUT ME

- Age: 34
- Middle School Computer Science Teacher
- Bachelor's Degree in Computer Science.
- Master's in Education.

## CORE GOALS

- Hands-on, practical learning.
- Integrating technology into her classroom.
- Provide students with resources they can use anywhere.
- Track each student's progress individually to offer tailored guidance.

## MOST IMPORTANT TASKS:

- Creating engaging lessons
- Tracking student progress
- Fostering an interactive learning environment.

## LEAST IMPORTANT TASKS:

- Routine administrative data entry
- minor system configurations.

## CORE CHARACTERISTICS:

- Classroom teaching
- Administrative duties.
- Extracurricular activities.

## PLACE OF WORK:

- Classroom
- Office
- Home (Lesson planning and resource development)

## TYPICAL CHALLENGES:

- Requires a platform that aligns with the school curriculum.
- Prefers a tool that offers a mix of tutorials, challenges, and real-world project scenarios.
- Wants a system that can generate reports on student performance to inform parent-teacher conferences and identify areas of improvement.

## SINGULARITIES:

- Always on the lookout for workshops or webinars on teaching methodologies.
- Often collaborates with other educators online to exchange resources and teaching strategies.
- Actively checks educational forums and platforms for reviews and recommendations on teaching tools.

## SOFTWARE EXPERTISE:

- Proficient in learning management systems, educational software, coding platforms, and data analysis tools.

## MAIN TASKS WITH THE SYSTEM SUPPORT:

- Browse through the modules and plan next month's lessons, assign specific modules to students, check the educator dashboard to review the progress of each student, and identify those who excelled and those who might need additional help.



# Sarah Martinez

HR Manager

- Age: 46
- Family: Married with two kids, ages 10 (Olivia) and 14 (Lucas).

## Core Characteristics

- Strong commitment to her family's well-being.
- Balances work responsibilities with quality family time.
- Values adaptability in the digital age.
- Strives to provide her children with well-rounded learning experiences.

## Core Goals

- Empower her children with enriching online educational platforms.
- Ensure her children's online safety and data privacy.
- Monitor and support her children's development without being intrusive.

## Typical Challenges

- Finding high-quality and engaging educational resources for her children.
- Managing screen time and fostering digital literacy.
- Balancing her own work schedule with family time.

## Most Important Tasks

- Empowering her children's learning experiences through online platforms.
- Ensuring her children's online safety and privacy.
- Monitoring her children's development and progress.

## Singularities

- Believes in lifelong learning and adapts to technological advancements.
- Encourages a balanced lifestyle that combines tech and offline activities.
- Concerned about her children's exposure to appropriate online content.

## Miscellaneous

- Enjoys family movie nights and reading books.
- Values weekend getaways for relaxation and bonding.
- Actively explores new apps for productivity and leisure.

## Working Situation

- HR Manager at a medium-sized corporation.
- Primary earner in her middle-class household.
- Engages with computers daily for work and personal use.

## Software Expertise

- Intermediate tech proficiency.
- Uses computers for work, owns a smartphone.

## Main Tasks with System Support:

- Ensuring her children access enriching online platforms.
- Implementing parental controls for screen time and communication.
- Communicating directly with platform administrators and educators.

## Least Important Tasks

- Engaging in unrelated activities on platforms.
- Participating in features that don't align with her children's learning goals.

# <EMILY JOHNSON>



Age: 27
Software Developer
Bachelor's Degree in Computer Science
5 years of working experience on building educational apps

## CORE CHARACTERISTICS:

- Embraces Teamwork
- Continuous Learning (Eager to learn new things)

## CORE GOALS:

- Professional development
- Create positive impact towards end users
- Work-life balance

## TYPICAL CHALLENGES:

- Struggles when changing project requirements constantly
- Concerned about ensuring the quality of codes and meeting the performance benchmarks
- Finds challenging while communicating with non-technical teams

## SINGULARITIES:

- Team player
- Innovative

## MOST IMPORTANT TASKS:

- Development and design of the "Learning Modules" feature, particularly on creating an interactive tutorial for Python loops.

## LEAST IMPORTANT TASKS:

- Checking logs for overnight system crashes or bugs.

## MISCELLANEOUS:

- Participate in Hackathons
- Loves open-source contributions
- Enjoys playing video games
- Blogs about new technology trends

## WORKING SITUATION:

- Full-time software developer

## PLACE OF WORK:

- Office
- Home

## SOFTWARE EXPERTISE:

- Proficient in several programming languages which is Python, Javascript, and React.
- Experienced educational apps builder and an expert user who is familiar with AWS Cloud Services.

## MAIN TASK WITH THE SYSTEM SUPPORT:

- Facilitate the development, testing and progression involved in creating the new "Learning Modules" feature.
- Coordinating with team members to track and manage tasks by ensuring the quality and stability of the code to provide more efficient version control.

# </EMILY JOHNSON>

## 2.5 Stakeholder Interview Outcomes

Interviewed a peer from another group (Daniel Lim) assuming the persona of an Educator. We held an unstructured interview with open-ended questions.

### 2.5.1 Information and Requirements Elicited

- The platform should have a dashboard for educators to track the progress of all students.
- The possibility to assign specific modules or challenges to students.
- Resources or guides that educators can use offline.

### 2.5.2 Scenario

An educator logs into CodeVenture before class. They assign a new module on Python loops to their students. After class, they check the progress dashboard to see how many students completed the assignment and their scores on related quizzes. The educator then uses this data to tailor the next in-person lesson.

This stakeholder analysis provides a detailed insight into the key individuals and groups associated with CodeVenture, ensuring the development process caters to their varied needs and preferences.

# 3. System Features

## 3.1 User Authentication and Profile Management

### 3.1.1 User Registration

Distinctive & Secure Onboarding:

- The platform should incorporate a robust user registration mechanism that mandates the exclusivity of usernames, ensuring no two individuals have the same identifier. Moreover, the system will integrate best practices in password security, including the enforcement of a password complexity algorithm. This ensures that every user has a distinctive and secure mode of access.
- ***Example***: When Amber initiates the registration process, she is prompted to input her desired username. The back-end algorithms check the database to validate the exclusivity of her choice. Following this, Amber is guided to set a password, where she's presented with specific criteria, such as length and the inclusion of alphanumeric characters, to ensure its strength.

### 3.1.2 Password Recovery

Secure Authentication Reset:

- The platform is to be equipped with a comprehensive, multi-faceted password recovery method, emphasising security. An integral part of this process would be email verification, which serves a dual purpose: authentication and security. Users will receive a verification link via email which will be time-constrained to prevent misuse.
- ***Example***: Dylan, realising he can't recall his password, selects the "Forgot Password" option. This triggers an automated email containing a link that will allow him to reset his password. This link has a finite validity period, ensuring that the reset option cannot be exploited indefinitely.

### 3.1.3 Profile Management

Transparent User Data Management:

- The system should offer a user-friendly interface enabling users to update their personal details effortlessly. An essential feature of this interface is the visual representation of the timestamp corresponding to the most recent update. This ensures transparency, instilling confidence in users about the integrity of their data.
- ***Example***: Following his recent relocation, Hanif updates his residential details on his profile. Post this change, he can immediately spot the timestamp, marking the exact date and time of the alteration.

## 3.2 Learning Modules

### 3.2.1 Content Categorisation

Structured Content Sequencing:

- It's imperative that the platform categorises learning content in a meticulous manner. This categorization, focused on specific constructs of the Python programming language, will guarantee a logical sequence in the delivery of content, fostering systematic and progressive learning.
- ***Example***: Octavia, aiming to deep dive into a specific topic, filters the content under the category "Loops". The system then showcases all lessons associated with this particular topic.

### 3.2.2 Interactive Learning

Engaging Learning Tools:

- To elevate the learning experience, each instructional module on the platform should be embedded with interactive tools. Features such as live coding environments or intuitive drag-and-drop tasks can enhance comprehension and make learning more engaging.
- ***Example***: While exploring the module on "Variables," Sasha encounters an in-built code editor. As she inputs her code, the platform offers real-time syntax highlighting, enhancing her coding experience

### 3.2.3 Real-time Feedback

Instantaneous Learning Evaluation:

- The system's capability to instantaneously evaluate user submissions is paramount. Immediate feedback, especially corrective suggestions in case of errors, can significantly enhance the learning curve, fostering a culture of iterative improvement.
- ***Example***: Isabelle attempts a task involving loop constructs but makes a syntax error. The platform's algorithms not only identify the flaw but also offer her a concise elucidation on the mistake, aiding her learning.

## 3.3 Challenges and Quizzes

Post-module quiz:

- After each module, the system shall present quizzes to test user understanding, in order to reinforce learning and assess understanding.
- ***Example:*** After Jeff finishes the "Functions" module, he takes a short quiz on the topic.

Retry quizzes:

- The system shall allow users to retry quizzes to improve scores, helping to promote mastery learning.
- ***Example****:* Troy didn't do well on her first attempt, so she reviews and retakes the quiz.

Hints & Solutions:

- The system shall provide hints or solutions after a certain number of incorrect attempts on challenges and aid to guide learning without causing excessive frustration.
- ***Example****:* Pierce struggles with a challenge, and after 3 failed attempts, a hint button appears.

## 3.4 Progress Tracking

Visual Learning Journey Overview:

- User progress is a vital metric, and the platform should visually represent this via a progress bar or an analogous tool. This visual aid offers users insights into their journey, indicating how much they've covered and what remains.
- ***Example***: Upon logging in, Harold is greeted with a visual representation indicating he has successfully covered 60% of the total course material.

Guided Learning Recommendations:

- To ensure continuity in the learning journey, the system should proactively recommend subsequent modules or topics based on a user's current progress.
- ***Example***: Kevin, having mastered the "Loops" section, is provided with a recommendation to delve into the "Conditions" module next.

## 3.5 User Interface

Centralised Learning Dashboard:

- A central dashboard will be a pivotal feature of the platform, offering users a consolidated view of their activities, progress, and tailored recommendations. This dashboard serves as a one-stop-shop for users to gain insights into their learning trajectory.
- ***Example***: On logging in, Lucy can effortlessly navigate to her dashboard, which showcases her recent accomplishments and offers suggestions on modules she should prioritise next.

Adaptive Cross-device Experience:

- Ensuring cross-device compatibility is crucial. The user interface should be adept at recognising the accessing device, whether it's a desktop, tablet, or mobile, and adjust its layout accordingly. This ensures a consistent and optimised user experience, irrespective of the device.
- ***Example***: Albert, choosing to access CodeVenture via his tablet, is presented with a layout that's perfectly tailored to his device's screen dimensions.

## 3.6 Stakeholder Views

Educators' Interface:

- Dedicated portals should be allocated for educators that enable a holistic perspective on the aggregate progression of their students. This design should facilitate educators to not only monitor the academic achievements but also offer intervention where needed, thus optimising the learning outcomes.
- ***Example***: Upon authentication, Teacher Nora is redirected to a specialised dashboard that provides a granular breakdown of her class's performance metrics, highlighting the areas of proficiency and those needing attention.

Administrative Console:

- A robust administrative interface is pivotal for the platform's efficient functioning. This control panel should empower admins with tools for meticulous user account management and a comprehensive content regulation mechanism, ensuring the platform operates within the defined regulatory and community guidelines.
- ***Example***: Admin Fred receives a notification regarding a user's inappropriate activity. Leveraging the control panel, he swiftly deactivates the reported account, upholding the platform's community standards.

## 3.7 Ethics and Accessibility

Inclusive Design:

- To ensure that the platform is universally usable, adaptive accessibility features must be ingrained. Such features, like adjustable font dimensions and compatibility with screen reading utilities, are essential to cater to users with diverse visual requirements and disabilities.
- ***Example***: Penny, faced with visual constraints, seamlessly toggles the interface to enlarge textual elements, thus enhancing her reading experience.

Data Privacy Compliance:

- Upholding ethical standards, the system must firmly commit to not archiving any user's personal information without obtaining a clear, informed consent. This practice ensures compliance with stringent data protection regulations and bolsters user trust.
- ***Example***: Quentin, during his onboarding process, is presented with a detailed consent form, explaining the nature and extent of data collection, requiring his explicit agreement before proceeding.

## 3.8 Feedback and Support

User-driven Improvements:

- A dedicated channel for users to highlight technical glitches or offer constructive feedback ensures that the system evolves in alignment with its user base's needs and experiences.
- ***Example***: Rachel, while navigating, encounters a rendering error. She promptly communicates this through the "Feedback" mechanism, aiding the tech team in identifying and rectifying the issue.

Self-help Resources:

- A comprehensive "Help" or "FAQ" repository should be accessible to users, addressing a wide array of common queries, thus ensuring users have immediate assistance at their fingertips.
- ***Example***: Steve, perplexed about the intricacies of progress tracking, finds a detailed explanation within the FAQ section, streamlining his user journey.

## 3.9 Performance and Reliability

Operational Consistency:

- The system architecture must be robust, aiming for an uptime benchmark of 99.5%. This ensures that users have almost uninterrupted access, barring minimal downtime for essential maintenance.

- ***Example***: Throughout a given calendar month, users experience a seamless performance, except during a pre-scheduled, brief system overhaul.

Load Resilience:

- The platform's back-end infrastructure should be geared to accommodate concurrent access by a significant user base, ensuring stability even during peak traffic scenarios.
- ***Example***: A major promotional campaign sees a surge in user logins. However, the platform, fortified for high concurrent user loads, functions without any latency issues.

## 3.10 Content Management

Dynamic Content Adaptability:

- The administrative privileges should extend to rigorous content management, including the provision to introduce, modify, or excise learning modules, ensuring the curriculum remains aligned with the latest advancements.
- ***Example***: Recognising a pivotal update in Python, admin Tara integrates a freshly curated module focusing on "Pattern Matching".

Data Preservation:

- Periodic automated backups of the entire repository of educational resources must be scheduled, safeguarding against potential data breaches or system failures.
- ***Example***: A weekly cron job ensures that the latest versions of all tutorials, quizzes, and associated resources are archived securely.

## 3.11 Community and Collaboration

Collaborative Learning Environment:

- A community hub should be integrated, fostering peer-to-peer interactions. Users can elucidate doubts, share insights, or engage in constructive discussions around specific topics or challenges.

- ***Example***: Odin, grappling with a coding challenge, initiates a discussion thread in the forum. Fellow learners offer diverse solutions, enriching his understanding.

Performance Motivators:

- Leaderboards or analogous tools can play a pivotal role in gamifying the learning experience, engendering a competitive spirit and driving user engagement.
- ***Example***: Vanessa, noticing her rank within the top echelons for the "Functions" module, is invigorated, setting her sights on even loftier academic goals.

User-generated Challenges:

- A segment of the platform should be dedicated to community-driven problem statements and solutions. This allows users to devise intricate or niche challenges, elevating the platform's content depth and range.
- ***Example***: Sahab, eager to delve deeper into recursion, feels the platform's challenges lack rigour. He navigates to the community segment, discovering a plethora of intricate problems curated by fellow learners.

These requirements, methodically segmented by feature sets, encapsulate the comprehensive functionalities the system aspires to deliver. As with any evolving digital device/application, these stipulations should undergo periodic refinement, absorbing stakeholder perspectives and undergoing detailed analyses to ensure clarity and completeness.

# 4. Non-Functional Requirements

### Description and Rationale

- Performance: Ensures the system remains responsive, even with many users, to provide a seamless learning experience.
- Usability: Facilitates learning by making the platform intuitive and easy to navigate.
- Reliability: Ensures the platform is dependable, fostering user trust.
- Security: Protects user data, especially crucial given the potential presence of minors.
- Availability: Guarantees constant access to learning resources.
- Maintainability: Allows for updates and improvements, ensuring the platform stays current with educational needs.
- Scalability: As the user base grows, the system must handle increased loads.
- Ethical and Inclusivity: Guarantees that the platform remains accessible, fair, and equitable for all users.

## 4.1 Performance

- The system's response time should not exceed 2 seconds under normal load conditions, to provide a smooth user experience.
- ***Example***: When Alice attempts a quiz, the results appear within 2 seconds.

## 4.2 Usability

- The platform should follow modern accessibility guidelines, to make the platform accessible for users with disabilities.
- ***Example:*** Text contrasts meet standards ensuring users with visual impairments can read content comfortably.

## 4.3 Reliability

- The system should achieve a Mean Time Between Failures (MTBF) of 1000 hours, to ensure platform dependability.

- ***Example:*** A server crash should be a rare occurrence, with long intervals in between.

## 4.4 Security

- All user passwords should be hashed, to protect user credentials.
- ***Example:*** Taraaf's password isn't stored as plain text but as a cryptographic hash.

## 4.5 Availability

- The system should achieve 99.5% uptime, excluding scheduled maintenance, to ensure users can access resources anytime.
- ***Example:*** Over a year, unscheduled downtime is less than 44 hours.

## 4.6 Maintainability

- Code should adhere to the modern coding standards, to simplify future maintenance and updates.
- ***Example:*** Developers follow guidelines on code structure and comments.

## 4.7 Scalability

- The system should handle annual user growth, without degradation of performance, to support a growing user base.
- ***Example***: If 10,000 students make use of CodeVenture in the current year, the system can support 11,500 users the next year without issues.

## 4.8 Ethics and Inclusivity

### 4.8.1 Cultural Sensitivity

- The content should be culturally sensitive and unbiased, to ensure fairness and inclusivity in learning.

- ***Example***: Quizzes don't include cultural references that might alienate users from diverse backgrounds.

### 4.8.2 Advertising

- The platform should not display any type of ads, to avoid commercial influences, especially given the young audience.
- ***Example:*** Even if there's potential for ad revenue, young learners are never exposed to ads.

### 4.8.3 Handling of User Data

- User data should not be shared with third parties without explicit consent, to maintain user trust and ethical standards.
- ***Example***: Yew Yi's performance data isn't shared with external platforms unless he agrees.

### 4.8.4 Language

- The platform should offer multiple language options, to cater to a global audience and promote inclusivity.
- Example: Taraaf, a non-native English speaker, switches to his mother tongue for ease of understanding.

### 4.8.5 Feedback

- The platform should have a feedback mechanism for content accuracy and sensitivity, to continually refine content and maintain inclusivity.
- ***Example:*** Edward, noticing a culturally insensitive example, flags it for review.

### 4.8.6 UI

- The system should have a simple and clear user interface, to make the platform user-friendly, especially for younger users.
- ***Example:*** Maya, a beginner, finds the layout intuitive and isn't overwhelmed.

### 4.8.7 Auditing and Standards

- Regular audits should be conducted to ensure the platform's ethical and inclusivity standards, to continuously uphold and refine the platform's ethical benchmarks.
- ***Example:*** A third-party reviews the content annually to check for any implicit biases.

## 4.9 Compatibility

- The system should be accessible across various devices and platforms, to ensure all users can access the platform
- ***Example:*** Daniel is able to access CodeVenture from 3 different devices, all running different operating systems.

# 5. Acceptance Criteria

## 5.1 User Registration

- Users can register with unique usernames that meet the defined security criteria.
- Passwords must meet the specified complexity criteria.
- CodeVenture should prevent duplicate usernames from performing account registration.

## 5.2 Password Recovery

- Users can initiate the password recovery process.
- An email containing a time limit verification link for password reset is sent upon password recovery request.

## 5.3 Profile Management

- Users can modify their personal details through an intuitive interface.
- CodeVenture displays the timestamp of the last update for each user profile modification.

## 5.4 Learning Modules

- Learning content is categorised under specific Python constructs.
- Modules incorporate interactive mechanisms (live coding, drag-and-drop exercises).
- User submissions within modules are promptly evaluated and provide corrective feedback.

## 5.5 Progress Tracking

- A progress bar or equivalent is displayed for each user.
- CodeVenture recommends the next module or topic based on user progress.

## 5.6 User Interface

- Users have a dashboard displaying recent activities, progress, and recommendations.
- CodeVenture interface's layout is responsive and adjusts to different device screens.

## 5.7 Stakeholder Views

- Educators have access to a view that tracks collective student progress.
- Admins have a control panel for managing user accounts and content.

## 5.8 Ethics and Accessibility

- CodeVenture includes font size adjustments and a screen reader-friendly mode.
- Personal data is not stored without explicit user consent.

## 5.9 Performance and Reliability

- CodeVenture maintains an uptime of 99.5%.
- Minimum of 10,000 users could be supported by the platform simultaneously.
- CodeVenture's MTBF should meet or exceed the 1000-hour threshold when under a typical load.

$$\text{MTBF} = \int_0^\infty R(t)dt$$

$\text{MTBF}$ = mean time between failures
$R(t)$ = reliability function

## 5.10 Feedback and Support

- Users can report bugs or provide feedback through a designated option.
- A "Help" or "FAQ" section is accessible to address common queries.

## 5.11 Content Management

- Admins can add, edit, or remove modules.
- Learning content is regularly backed up to prevent data loss.

## 5.12 Community and Collaboration

- Users can participate in discussions in a community forum.
- Top-performing learners will be showcased on a leaderboard.
- Users can create and solve problems in the community problems section.

## 5.13 Security

- All users' passwords are stored as cryptographic hashes in the database.
- Hashing algorithm implemented in CodeVenture is industry standard and cannot be retrieved easily through reverse-engineering.

## 5.14 Compatibility

- Users can access CodeVenture through various devices including desktops, tablets, mobile phones and different browser platforms such as Chrome, Firefox and Safari.
- The compatibility of different operating systems are tested or examined to ensure consistent functionality of CodeVenture.

## 5.15 Challenge and Quiz

- Users are notified by CodeVenture upon their quiz completion and their total score.
- Users could utilise the hints provided to help them solve the quizzes.
- Users have the option to retry the quizzes to improve their understanding or scores, each set of questions is new to avoid users memorising the answers.
- If users continue to struggle and fail after several attempts, a "solution" button will appear and comprehensive explanations of how to solve the questions will be provided.

# 6. Requirements Quality Analysis

To ensure the highest quality in our requirements, it's essential to assess them based on key quality attributes. This helps in the early identification of potential problems and rectifying them, thus saving time and resources in the later stages of the project.

## 6.1 Key Quality Attributes to Assess

- Clarity: Each requirement should be clear and easily understood without ambiguity.
- Consistency: Requirements shouldn't contradict one another.
- Completeness: No essential part of the software solution should be omitted.
- Feasibility: It must be possible to implement each requirement within the current technology and budget.
- Relevance: Each requirement should be relevant to the project goal.
- Testability: It should be possible to verify each requirement once implemented.
- Traceability: Each requirement should be uniquely identifiable and traceable throughout its lifecycle.

## 6.2 Common Requirements Defects

- Omission: Missing key requirements.
- Ambiguity: Unclear descriptions leading to multiple interpretations.
- Incorrect Fact: Requirements based on erroneous information.
- Inconsistency: Conflicting requirements.
- Not Feasible: Requirements that cannot be implemented given current constraints.

## 6.3 Analysis and Improvements

**Clarity:** We used simple language, avoiding jargon wherever possible. Each requirement was reviewed by multiple team members to ensure it was easily understandable. Initially, "The platform should adhere to accessibility criteria" was rewritten to "The platform should follow the Web Content Accessibility Guidelines (WCAG) 2.1 for accessibility" for better clarity.

**Consistency:** Cross-referenced all requirements to ensure they didn't contradict each other. We ensured that the requirement for no ads on the platform was consistent with the goal of user safety and trust.

**Completeness:** Conducted brainstorming sessions with stakeholders and reviewed the requirements multiple times to ensure no key requirements were omitted. Added a requirement for multi-language support after stakeholder feedback.

**Feasibility:** Consulted with technical experts to ensure all requirements were feasible. While the initial requirement was for a response time of 1 second, after consultations, it was revised to 2 seconds for better feasibility without compromising user experience.

**Relevance:** Prioritised requirements based on their relevance to the project's goals. Removed a potential requirement for social media integration, as it was deemed not directly relevant to the educational objective of the platform.

**Testability:** Each requirement was written in a way that its fulfilment can be tested. "The system should be fast" was refined to "The system's response time should not exceed 2 seconds under normal load conditions."

**Traceability:** Ensured each requirement is uniquely identifiable and can be tracked throughout its lifecycle. Each requirement was labelled with a unique ID to maintain traceability. By diligently analysing the requirements for these attributes and common defects, we enhanced the quality of our requirements, making them clearer, more consistent, and more aligned with the project's goals and constraints.

# 7. Document Quality

This section highlights details that we find relevant for improving the system's overall quality, the readability of the requirements document, or the overall understanding of the project.

## 7.1 Document Structure

- **Table of Contents:** A detailed table of contents that allows stakeholders to quickly jump to different sections of the document.
- **Glossary:** Define all technical terms, abbreviations, and project-specific jargon for clarity.

## 7.2 Visual Aids

- **Flowcharts:** For system flow, particularly when describing how different requirements integrate or when describing complex processes.
- **Diagrams:** Use UML or ERD diagrams where applicable to provide a visual representation of the system's architecture or data relationships.
- **Screenshots:** Mock-ups or wireframes of the interface, highlighting where each requirement would be visible to the end-user.

## 7.3 Requirement Categorisation

- **Requirement IDs:** Unique identification numbers for each requirement to enhance traceability.
- **Requirement Tags:** Use tags to categorise requirements (e.g., usability, security, performance) so stakeholders can easily filter and focus on areas of interest.

## 7.4 Examples and Scenarios

Providing real-world examples or user scenarios for complex requirements can make them more understandable.

## 7.5 Versioning

Include a version number and a changelog at the beginning of the document. This ensures that all stakeholders are referencing the most up-to-date document and can track changes over time.

## 7.6 Feedback Mechanism

Incorporate a section where stakeholders can note their feedback or questions. This ensures an iterative refinement process.

## 7.7 Stakeholder Profiles

A section briefly describing the different stakeholders and their potential interests in the project. This ensures that when reading, each stakeholder knows which parts of the document are most relevant to them.

## 7.8 Document Formatting

- Use clear headers and subheaders.
- Employ bullet points for easy readability.
- Highlight or box key points or critical requirements.
- Consistent font styles and sizes.
- Colour code or label different categories of requirements.

## 7.9 Document Summary

After an exhaustive analysis and iterative refinement process, we present this comprehensive requirements specification document for CodeVenture. Our goal has been to create a platform that makes the learning experience both enriching and enjoyable for young coders. We've kept user experience, security, and accessibility at the forefront of our considerations, ensuring that our platform is welcoming to all.

By collaborating closely with various stakeholders and integrating their invaluable feedback, we believe we've set a solid foundation for the development phase. We're excited about the potential of CodeVenture and are confident that, once realised, it will serve as a beacon for educational platforms worldwide. As we transition into the next phases of the project, this document will serve as a guiding light, ensuring that our vision for CodeVenture remains consistent and clear.